LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

Module Code & Module Title:

CS4051NT Fundamentals of Computing

Assessment Weightage & Type:

70% Individual Coursework

Year and Semester:

2025 Spring, 2nd Semester

Student Name: Manoj Katuwal

London Met ID: 24045922

College ID:NP05CP4A240306

Assignment Due Date: May 14,2025

Assignment Submission Date: May 14, 2025

Submitted To: Ajayraj Bhattarai

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

24045922 Manoj Katuwal

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:95768832

Submission Date
May 14, 2025, 8:25 AM GMT+5:45

Download Date
May 14, 2025, 8:40 AM GMT+5:45

File Name
24045922 Manoj Katuwal

File Size
18.5 KB

28 Pages

2,732 Words

14,709 Characters

16% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **47** Not Cited or Quoted 16%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
16% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 47 Not Cited or Quoted 16%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
16% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	
islingtoncollege on 2025-05-12		3%
2	Submitted works	
islingtoncollege on 2025-05-13		3%
3	Submitted works	
islingtoncollege on 2025-05-13		1%
4	Submitted works	
ESoft Metro Campus, Sri Lanka on 2025-04-27		1%
5	Submitted works	
islingtoncollege on 2025-05-13		1%
6	Submitted works	
Study Group Australia on 2012-12-02		<1%
7	Submitted works	
Asia Pacific University College of Technology and Innovation (UCTI) on 2024-11-04		<1%
8	Submitted works	
Edith Cowan University on 2024-08-22		<1%
9	Submitted works	
University of Technology, Sydney on 2006-05-05		<1%
10	Submitted works	
Asia Pacific University College of Technology and Innovation (UCTI) on 2025-01-19		<1%

turnitin Page 3 of 32 - Integrity Overview Submission ID trn:oid:::3618:95768832

stomer buy 4 products and get 1 free then the c

Table of Contents

Tables of Figures:

Table of Tables

1 Introduction

1.1 Introduction of Project

This project was given as the coursework of 1st year students based on real-life scenario of local vendor that sells beauty and skin care products which was named as WeCare. The store sells all types of skin curing disease like which contain salicylic acid, minoxidil acid and many more and beauty products like sunscreen, facewash, hair gel and many more. To manage the products available in a store a simple management system was build in this project. The store has also a given offer Buy 3 get 1 free to increase the more customer. The program also includes the following function that when customer buy 3 products and get 1 free then the stock automatically updates the remaining products as the amount of money only recorded as only included the selling product not the free one.

The application was developed using python and follows the basic data structure for collection and store of the data. The data of the products stored into the txt file which can update, read and write as needed.

2 Aims and Objective

Main aim of this program is to develop python-based system to manage the products sales and stock for a local vendor. This project helps me to build my strong concept regarding python file handling, exception, function, loops, dictionary, list etc. I learned to deal with the real-world problem scenario which increase and build my confidence level.

The Objective of this program are as follows:

- To develop a simple management system for a local vendor to manage the products.
- To arrange the stock of the product which was sold.
- To manage the sell product while offer “Buy 3 get 1 free” was given.
- To give access to the shop owner to change the price if needed.

3 Technology used:

3.1 Integrated Development and Learning Environment (IDLE)

IDLE is a default editor that comes with the python. It is a perfect tool for learning the python for beginners. IDLE provides a simple graphical user interface for writing, editing, running python code, syntax highlighting error which helps in debugging the code and fix it. IDLE is so beginner friendly, suitable for small project, easy to write, easy to debug so I have chosen the IDLE (Integrated Development and Learning Environment) for my first ever python project of my Fundamental of Computing project.

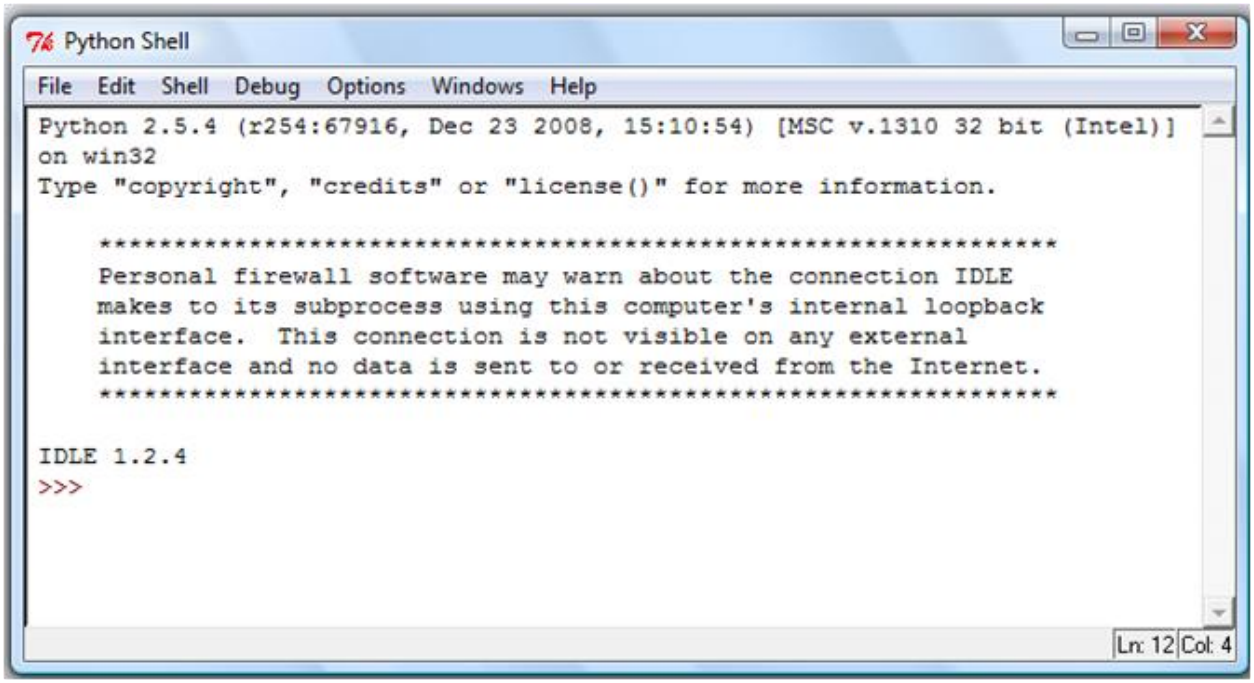


Figure 1: IDLE

4 Data Structure

In this project, data structure are used to store, manage and access product information like product id, name, brand, origin, etc. Primitive data types like string, integer, float are used to hold the value like product name and prices. Non-Primitive data types like dictionary and list are used to store the complex information of the products as dictionary store the key-value pairs and list are used to hold all production as collection.

DATA STRUCTURE	TYPE	DESCRIPTION	EXAMPLE
String	**Primitive**	**Stores text data for names and categories**	**product['name'] = "Aloe Vera Gel"**
Integer	Primitive	Handles whole numbers for quantities	product['quantity'] = 50
Float	**Primitive**	**Manages decimal numbers for pricing**	**product['price'] = 12.99**
Dictionary	Non-Primitive	Key-value pairs for structured product records	product = {'name', 'price'}
List	**Non-Primitive**	**Ordered collection for multiple product entries**	**products = [{product1}, {product2}]**

Table 1:Data Structure

5 Algorithm

An algorithm is a step by step instructions used to solve the problem or to perform a special task. In this WeCare Inventory system, the algorithm handles user interface satisfaction, manage customer purchases, restock the product and create the invoice of both sale and restock.

Step 1: Start the program.

Step 2: Read product data from products.txt file using read.read_products() method.

Step 3: If product.txt file doesn't exist, it create a new file name products.txt.

Step 4: Store the data into the list name products.

Step 5: Display the main menu with options:-

1. Display Available Products.
2. Process Sales Transaction.
3. Process Restock Transaction.
4. Exit

Step 6: Prompt user to enter a choice(0-3).

Step 7: If invalid number is selected, it display a message "Invalid choice. Please try again" and return to step 5.

Step 8: If choice is 0 then it return to step 26.

Step 9: If choice is 1(Display Available Products).

1. Call read.display_Products() method.
2. Show products details in a well structed table which show the id, product name, brand, origin, stock, cost price and selling price.
3. Prompt user to like to do another operation or not(y/n).
4. If y then it return to step 5.

Step 10: If choice is 2 (Process Sales Transaction).

1. Call read.display_products() method.
2. Show the products details in a well structed format table.
3. Prompt for customer name.
4. Prompt user to select the product id for the purchase of the product.
5. If the user select the invalid id then it display the message " Value must be the present in the product. Please try again" and ask user again to select the id.

6. If user select 0, then it display the message “No items selected. Transaction cancelled” and ask user again to like to do another operation or not(y/n).
7. If yes then it return to step 9.

Step 11: Prompt user to the quantity of the product to be purchase.

Step 12: Apply “buy 3 get 1 free policy” and update the quantity in the product list.

Step 13: For the selected product, check if the stock is greater then 0. If not display a message “out of stock” to purchase.

Step 14: Add item to the cart and purchased item list with sub-total and total amount.

Step 15: Generate sales invoice using write.generate_sales_invoice() method along with customer name, purchased items, free items and total amount customer has to pay.

Step 16: Update products.txt file with the modified quantities.

Step 17: Ask user after invoice creation that would you like to perform another operation or not (y/n).

Step 18: If y then return to step 5.

Step 19: If n then to return to step 26.

Step 20: If choice is 3 (Process Restock Transaction).

1. Display the product list along with their id, name, origin, stock, cost price and selling price of product.
2. Prompt for supplier name.
3. Prompt user to select the number of product id which the user wants to restock.
4. If 0 is selected then return the message “No items selected. Restock cancelled”.

Step 21: If the id of the product is selected then ask the user the quantity to be restock for the product.

Step 22: Ask the user has the cost price has changed or not.

Step 23: If the cost price has changed then you can update the cost price.

Step 24: Generate restock invoice along with the product name, brand, cost, quantity and amount.

Step 25: Update the restock amount in the product.txt file.

Step 26: If choice is 0 (Exit).

1. Display the message “Thank you for using WeCare Skin Care Product System. Goodbye!” and exit the program.

6 Flow Chart

Flow Chart is a visual representation of the algorithm, provide a visual way to understand how the program works and makes easier in designing and explaining structure of the program. For my project, WeCare Inventory Management system, I have created to show the sequence of my operation in my project.

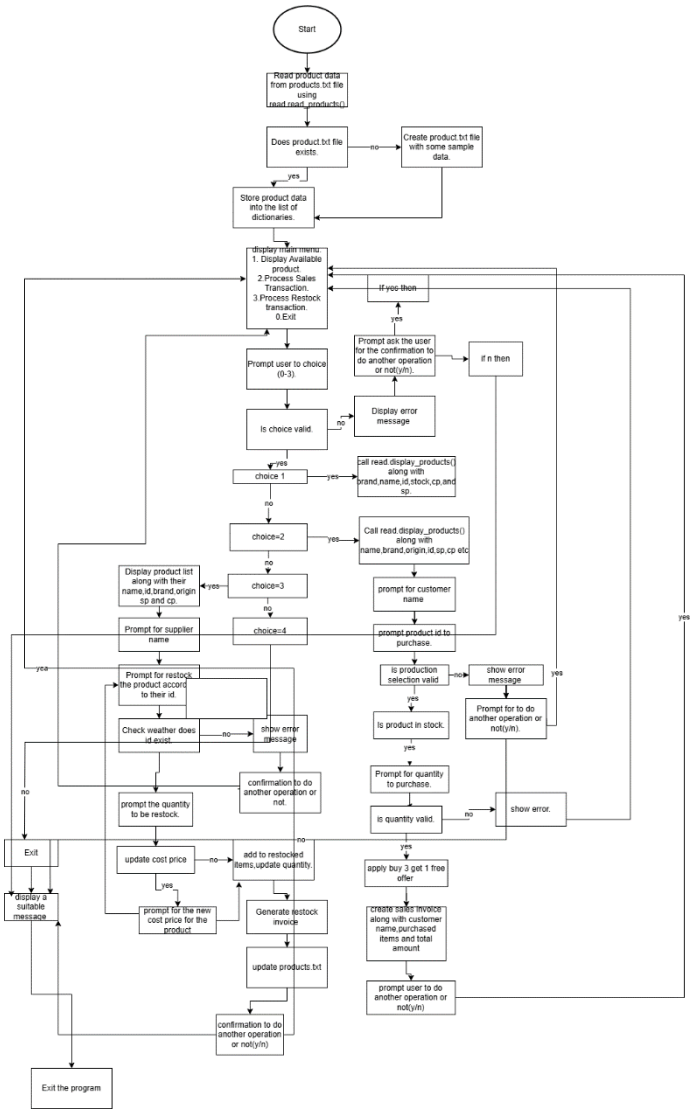


Figure 2: Flow Chart

7 Pseudocode

Pseudocode is a simplified half English, half code of writing a program to express the program logic. It helps the developer for planning and explaining the code without the code syntax. For my project, I have created the whole program module pseudocode to explain and express the program logic.

7.1 Read module pseudocode

```
START MODULE
FUNCTION read_products()
        INITIALIZE empty product list
        TRY
        OPEN products.txt file for reading
        FOR each line in file
                IF line has space remove it
                CREATE product dictionary with:
                Name = parts[0]
                Brand=parts[1]
                Quantity=converts parts[2] into int
                Cost_price= convertd part[3] into float
                Origin=parts[4]
        APPEND product to product list
                ELSE
                        DISPLAY warning for invaid format
        ClOSE File
RETURN products list
CATCH FileNotFoundError
        DISPLAY error message
        RETURN empty list
CATCH other exception
        DISPLAY error message
```

```
        RETURN empty list
END FUNCTION
FUNCTION display_products()
        IF product is not found
        DISPLAY “No Products Available” message
        RETURN
        DISPLAY a line separator
        DISPLAY the header along with the ID, Product, Brand, Origin, Stock, Cost(Rs)
        and Sale(Rs)
        DISPLAY a line separator
        INITALIZE Index = 1
        FOR each product in products
        CALCULATE selling_price = product[‘cost_price’] *2
        DISPLAY index, product name, product brand, product origin, product quantity,
        product cost_price and product selling_price.
        INCREMENT index by 1
        DISPLAY a line separator
END FUNCTION
END MODULE
```

7.2 Write Module Pseudocode

START write module

FUNCTION write_products(products):

TRY

OPEN product.txt file for writing

FOR each product in products

CREATE formatted line with product name, product brand, product quantity, product cost_price and product_origin.

WRITE line to file

CLOSE file

RETURN true

CATCH any exception

DISPLAY error message

FUNCTION generate_sales_invoce(customer_name, items, total_amount)

GENERATE timestamp

CREATE filename with customer_name and timestamp

TRY:

OPEN filename.txt file for writing as invoice

WRITE header

WRITE invoice details like date, customer_name, invoice number

WRITE table header with columns: Product, Brand, Price, Qty, Free, Amount

FOR each item in items:

WRITE formatted row with item name, item brand, item selling_price, item quantity, item free, item amount

END FOR

WRITE formatted line for footer

WRITE total amount

DISPLAY thank u message

CATCH any exception

DISPLAY error message

RETURN none

FUNCTION generate_restock_invoice(supplier_name, items, total-amount)
GENERATE timestamp
CREATE filename with supplier name and timestamp
TRY:
    OPEN filename for writing as invoice
        WRITE header
        WRITE invoice details like Date, Supplier name, and purchased order date
        WRITE table header which includes product, brand, cost, qty and amount
         FOR each item in items
            WRITE formatted row with item name, item brand, item cost price, item quantity and amount.
         END FOR
    WRITE formatted line for footer
    WRITE total amount
    DISPLAY thank u message
CLOSE file
RETURN filename
CATCH any exception
DISPLAY error message
RETURN none

7.3 Operation Module Pseudocode

```
    FUNCTION get_valid_integer_input (prompt, min_value=0, max_value=None)
            WHILE true
                TRY
                    GET input value from user
                    IF value < min_value:
                        DISPLAY error message
                    ELSE IF max_value is not null AND value > max_value
                        DISPLAY error message
                    ELSE
                        RETURN value
            END FUNCTION
                    CATCH ValueError
                        DISPLAY error message

     FUNCTION process_sales_transaction(products):
            CALL display_products(products) method from read module
            DISPLAY products
       WHILE customer name is empty
        GET customer_name from user
        IF empty
           DISPLAY error message
        END IF
END WHILE
END FUNCTION

INITALIZE empty sale_item list
INITIALIZE total_amount=0
```

```
WHILE True
    GET product choice (0 to finish)
If choice is 0
  If saale_itemsA is empty
    DISPLAY cancellation message
    RETURN false
END IF
BREAK
END IF
GET product from products using choice
If product.quantity < =0
   DISPLAY out of stock message
    CONTINUE
END IF

GET quantity (max product.quantity)
CALCULATE selling_price = product.cost_price *2
CALCULATE free_items = quantity /3
CALCULATE total_items = quantity + free items
CALCULATE item_amount = quantity * selling_price
ADD item_amount to total_amount
ADD item to sale_item with details
UPDATE product.quantity by subtracting total_items
  DISPLAY confirmation message
END WHILE

GENERATE sales invoice
IF invoice generated successfully
     UPDATE products file
      DISPLAY success message with total_amount and free_items
   RETURN true
```

```
        END IF
        DISPLAY failure message
        RETURN false
END FUNCTION

   FUNCTION process_restock_transaction(products)
    DISPLAY products
   WHILE supplier_name is empty
    GET supplier_name from user
 IF empty
     DISPLAY error message
END IF
END WHILE

INITIALIZE empty restock_items list
INITIALIZE total_amount = 0

WHILE true
    GET product choice (0 to finish)
    IF choice is 0
      IF restock_items is empty
          DISPLAY cancellation message
          RETURN false
    END IF
     BREAK
   END IF

GET product from products using choice
GET quantity
SET new_cost_price = product.cost_price
IF users confirms price change
```

```
    GET new_cost_price (must be positive)
 END IF
CALCULATE item_amount = quantity *new_cost_price
ADD item_amount to total amount.
ADD item to restock_items with details
UPDATE product.quantity by adding quantity
UPDATE product_cost_pricee if changed
     DISPLAY confirmation message
END while

GENERATE restock invoice
IF invoice generated successfully
UPDATE products file
  DISPLAY  success message with total_amount
  RETURN true
END IF
   DISPLAY failure message
RETURN false
END FUNCTION
END MODULE
```

7.4 Main Module Pseudocode

```
START main module
  FUNCTION display_menu():
    DISPLAY formatted menu with options:
      1. Display Products
      2. Process Sales
      3. Process Restock
      0. Exit
  END FUNCTION

  FUNCTION main()
    DISPLAY welcome message
    WHILE true:
      READ products
      IF products is empty
       DISPLAY error message
       IF user doesn't want to retry
          BREAK
        END IF
        CONTINUE
        END IF
DISPLAY menu
TRY:
    GET user choice
    IF choice is 0
        DISPLAY goodbye message
     BREAK
     ELSE IF choice is 1
        DISPLAY products
     ELSE IF choice is 2
       PROCESS sales transaction
```

```
        ELSE IF choice is 3
            PROCESS restock transaction
        ELSE
            DISPLAY invalid choice message
        END IF
        CATCH ValueError
            DISPLAY invalid input message
        END try
        IF user don't want to continue
            DISPLAY good bye message
            BREAK
        END IF
    END WHILE
    END FUNCTION
END MODULE
```

8 Testing

8.1 Display product

PROCESS	OUTPUT
Objective	To display the products
Action	Run the program and select 1
Excepted output	Should display product id, product, brand, origin, selling price and cost price
Actual output	Display program id, product, brand, origin, selling price and cost price
Result	The test was successful.

Table 2: test 1

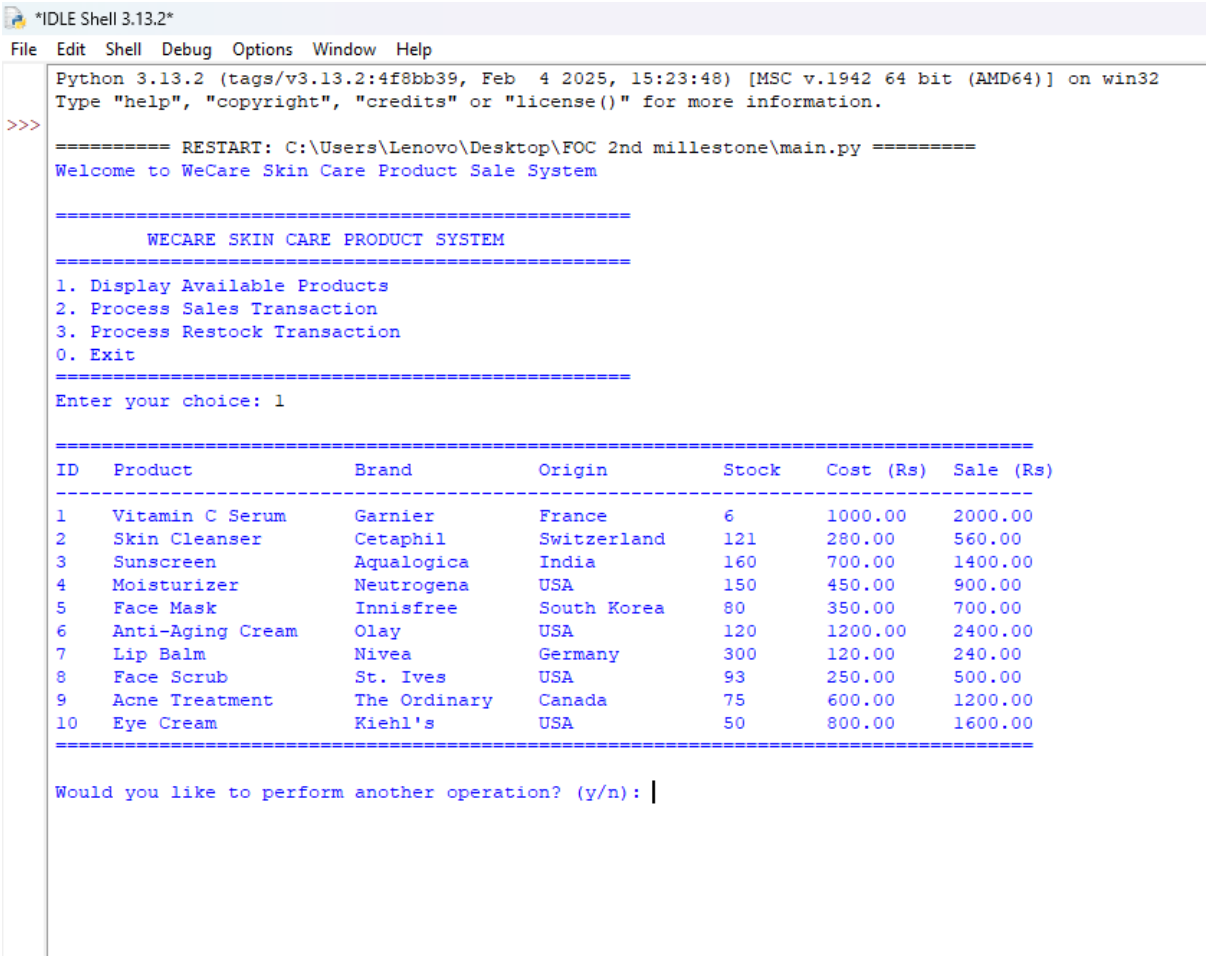


Figure 3:Display Product test

8.2 Sell product with invoice test

PROCESS	OUTPUT
Objective	To sell product with invoice file
Action	Run the product and choose 2 , enter the customer name, and select the product which whom you want to buy, enter the quantity of product, select 0 to finish the operaition
Excepted output	Product could be sell and generate the sale invoice.
Actual output	Product has been sell and generate sale invoice too.
Result	The test has been successful.

Table 3: sell product test

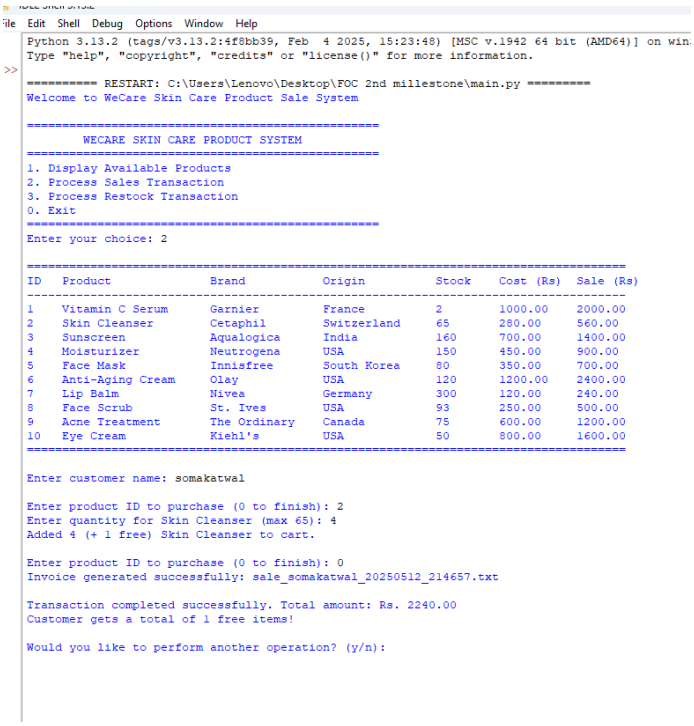


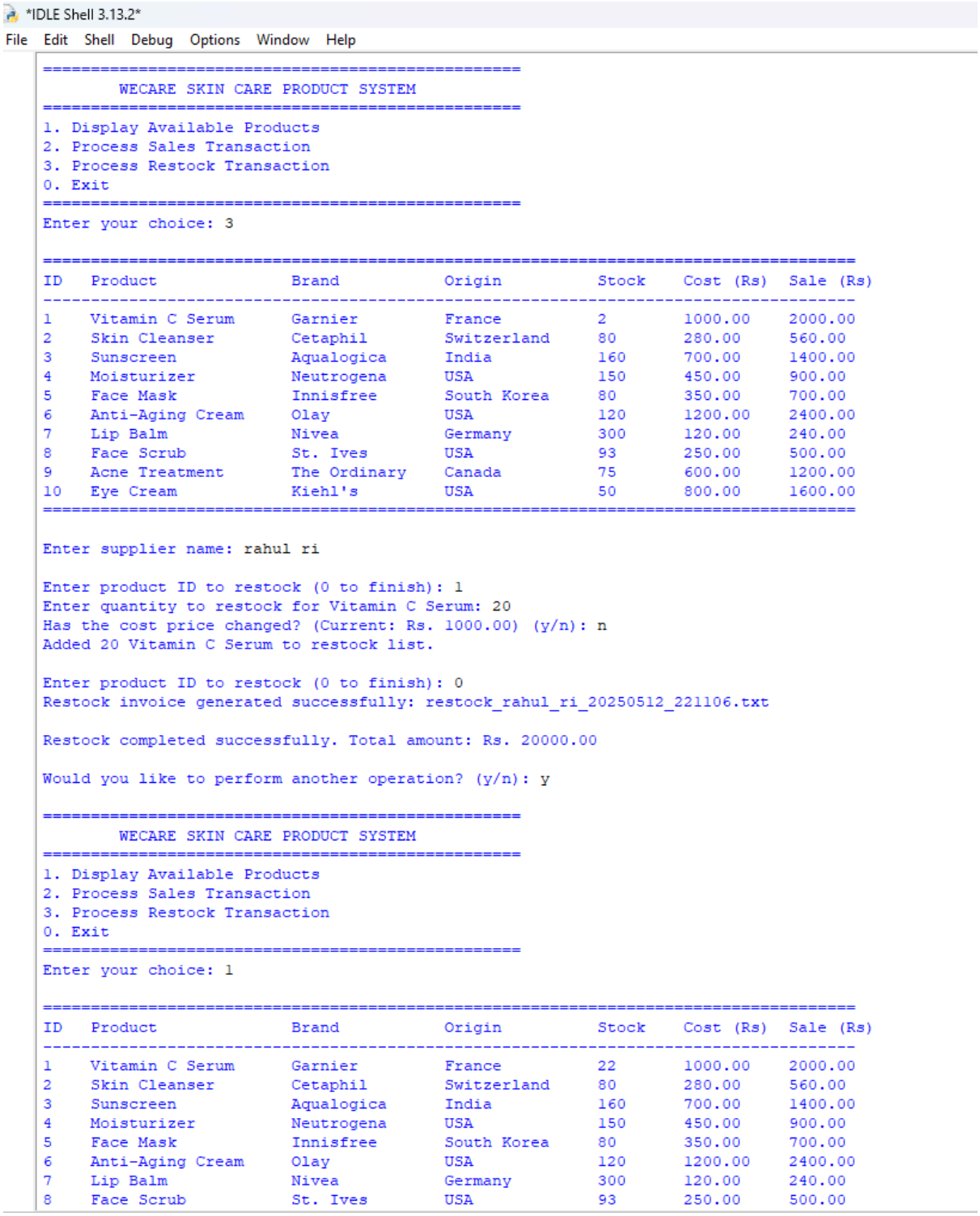
Figure 4:Sell test

Figure 5: Sell invoice

8.3 Buy product with invoice

PROCESS	OUTPUT
Objective	To buy the product.
Action	Run the program, select 3,enter supplier name, enter quantity and enter 0 to finish restocking.
Excepted output	Display buy product in the table and generate restock invoice.
Actual output	Display buy product in the table and generate restock invoice.
Result	The test was successful.

Table 4: buy product test



```
*IDLE Shell 3.13.2*
File  Edit  Shell  Debug  Options  Window  Help
================================================
        WECARE SKIN CARE PRODUCT SYSTEM
================================================
1. Display Available Products
2. Process Sales Transaction
3. Process Restock Transaction
0. Exit
================================================
Enter your choice: 3

=============================================================================================
ID   Product              Brand           Origin          Stock    Cost (Rs)  Sale (Rs)
---------------------------------------------------------------------------------------------
1    Vitamin C Serum      Garnier         France          2        1000.00    2000.00
2    Skin Cleanser        Cetaphil        Switzerland     80       280.00     560.00
3    Sunscreen            Aqualogica      India           160      700.00     1400.00
4    Moisturizer          Neutrogena      USA             150      450.00     900.00
5    Face Mask            Innisfree       South Korea     80       350.00     700.00
6    Anti-Aging Cream     Olay            USA             120      1200.00    2400.00
7    Lip Balm             Nivea           Germany         300      120.00     240.00
8    Face Scrub           St. Ives        USA             93       250.00     500.00
9    Acne Treatment       The Ordinary    Canada          75       600.00     1200.00
10   Eye Cream            Kiehl's         USA             50       800.00     1600.00
=============================================================================================

Enter supplier name: rahul ri

Enter product ID to restock (0 to finish): 1
Enter quantity to restock for Vitamin C Serum: 20
Has the cost price changed? (Current: Rs. 1000.00) (y/n): n
Added 20 Vitamin C Serum to restock list.

Enter product ID to restock (0 to finish): 0
Restock invoice generated successfully: restock_rahul_ri_20250512_221106.txt

Restock completed successfully. Total amount: Rs. 20000.00

Would you like to perform another operation? (y/n): y

================================================
        WECARE SKIN CARE PRODUCT SYSTEM
================================================
1. Display Available Products
2. Process Sales Transaction
3. Process Restock Transaction
0. Exit
================================================
Enter your choice: 1

=============================================================================================
ID   Product              Brand           Origin          Stock    Cost (Rs)  Sale (Rs)
---------------------------------------------------------------------------------------------
1    Vitamin C Serum      Garnier         France          22       1000.00    2000.00
2    Skin Cleanser        Cetaphil        Switzerland     80       280.00     560.00
3    Sunscreen            Aqualogica      India           160      700.00     1400.00
4    Moisturizer          Neutrogena      USA             150      450.00     900.00
5    Face Mask            Innisfree       South Korea     80       350.00     700.00
6    Anti-Aging Cream     Olay            USA             120      1200.00    2400.00
7    Lip Balm             Nivea           Germany         300      120.00     240.00
8    Face Scrub           St. Ives        USA             93       250.00     500.00
```

Figure 6:Product by test

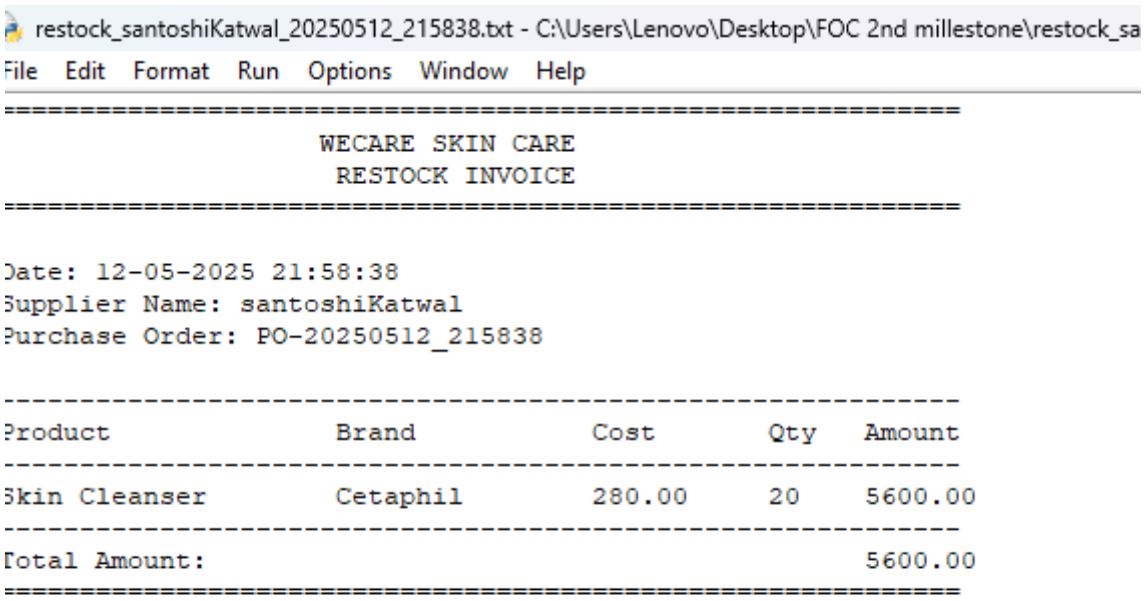


Figure 7:Restock invoice

8.4 Negative input for quantity (Restock Product Test)

PROCESS	OUTPUT
Objective	To verify the system handles negative value of quantity in restocking product.
Action	Run the program, select option 3 (process restock transaction), enter supplier name, select product id, and input the negative quantity.
Excepted output	Should give error message and ask user again to enter the quantity again.
Actual output	Give error message and ask user again to enter the quantity again.
Result	Successfully handled the program

Table 5: Restock negative input test

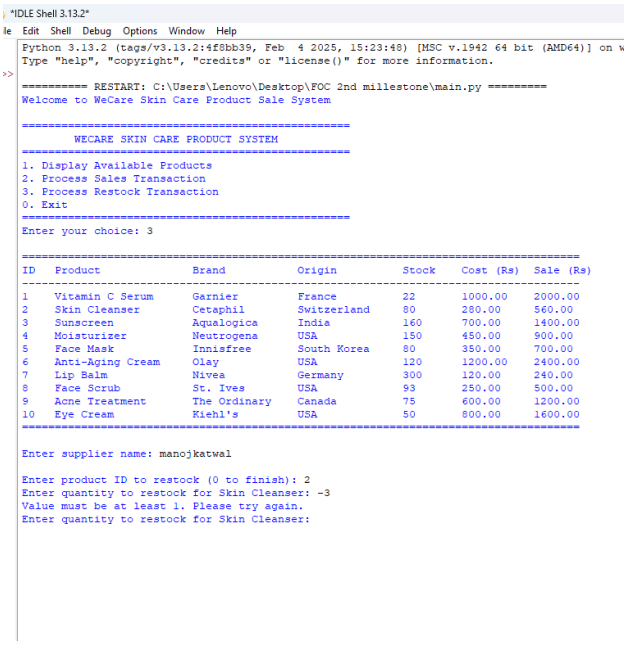
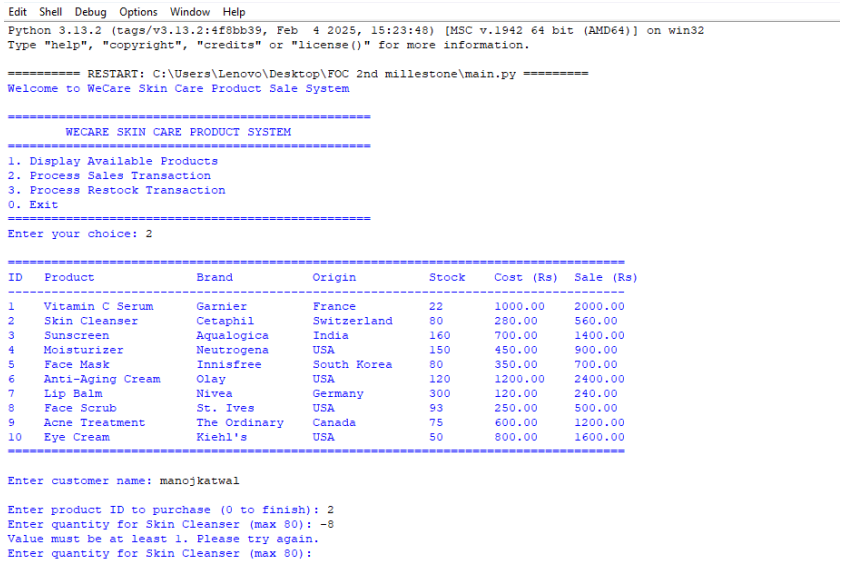


Figure 8:Negative value test in restock qunatity

8.5 Negative input for quantity (Sales Product Test)

PROCESS	OUTPUT
Objective	To verify the program handles negative value in quantity in selling product.
Action	Run the program, select option 3(process sales transaction), enter customer name, select product id and input negative value in quantity.
Excepted output	Should give error message and ask user to enter the quantity again.
Actual output	Give error message and ask user to enter the quantity again.
Result	Successfully handled by the program

Table 6: negative input for quantity in sales transaction



```
Edit  Shell  Debug  Options  Window  Help
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

========== RESTART: C:\Users\Lenovo\Desktop\FOC 2nd millestone\main.py =========
Welcome to WeCare Skin Care Product Sale System

================================================
        WECARE SKIN CARE PRODUCT SYSTEM
================================================
1. Display Available Products
2. Process Sales Transaction
3. Process Restock Transaction
0. Exit
================================================
Enter your choice: 2

================================================================================
ID   Product              Brand           Origin          Stock    Cost (Rs)  Sale (Rs)
--------------------------------------------------------------------------------
1    Vitamin C Serum      Garnier         France          22       1000.00    2000.00
2    Skin Cleanser        Cetaphil        Switzerland     80       280.00     560.00
3    Sunscreen            Aqualogica      India           160      700.00     1400.00
4    Moisturizer          Neutrogena      USA             150      450.00     900.00
5    Face Mask            Innisfree       South Korea     80       350.00     700.00
6    Anti-Aging Cream     Olay            USA             120      1200.00    2400.00
7    Lip Balm             Nivea           Germany         300      120.00     240.00
8    Face Scrub           St. Ives        USA             93       250.00     500.00
9    Acne Treatment       The Ordinary    Canada          75       600.00     1200.00
10   Eye Cream            Kiehl's         USA             50       800.00     1600.00
================================================================================

Enter customer name: manojkatwal

Enter product ID to purchase (0 to finish): 2
Enter quantity for Skin Cleanser (max 80): -8
Value must be at least 1. Please try again.
Enter quantity for Skin Cleanser (max 80):
```

Figure 9:Negative value test in sales transaction

8.6 Quantity more than the stock available

PROCESS	OUTPUT
Objective	To ensure the system prevent customer for purchasing more quantity then is available in stock.
Action	Run the program, choose option 2 (Process Sales Transaction), enter customer name, choose a product id with low stock and attempt to purchase more then the stock.
Expected output	Should give error message and ask user to enter the quantity again.
Actual output	Give error message and ask user to enter the quantity again.
Result	Successfully handled by the program.

Table 7: Quantity more than the stock available

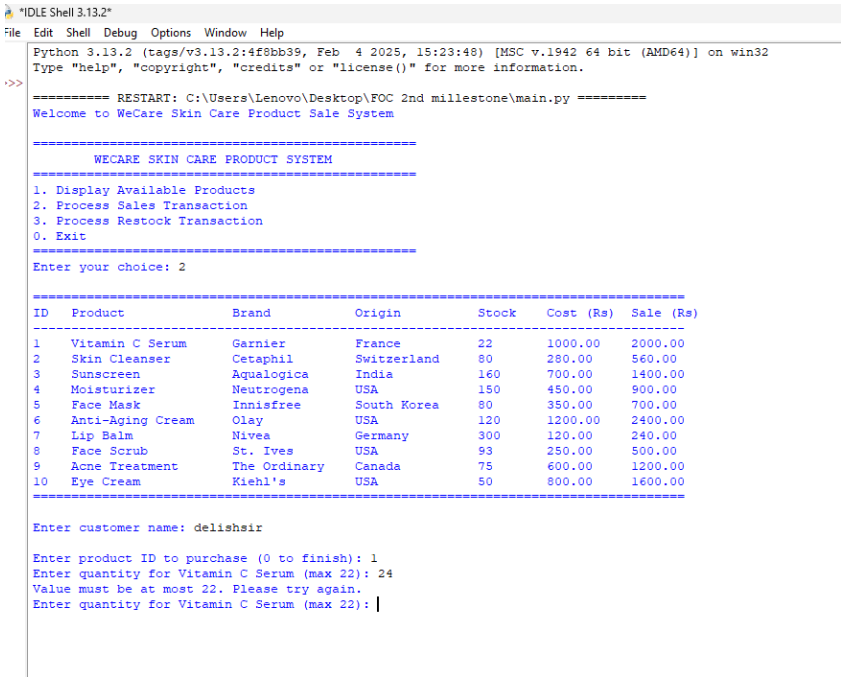


Figure 10:Qunatity test for more than the stock available

8.7 User entering product id that doesn't exist

PROCESS	OUTPUT
Objective	Verify that the system detects invalid product id and should display error message.
Action	Run the program, choose option 2 (Process Sales Transaction), enter customer name, chose a product id that doesn't exist.
Expected output	Should give error message and ask user again to enter the quantity.
Actual output	Give error message and ask user again to enter the quantity.
Result	Successfully handled the program.

Table 8:user entering product id that does not exist

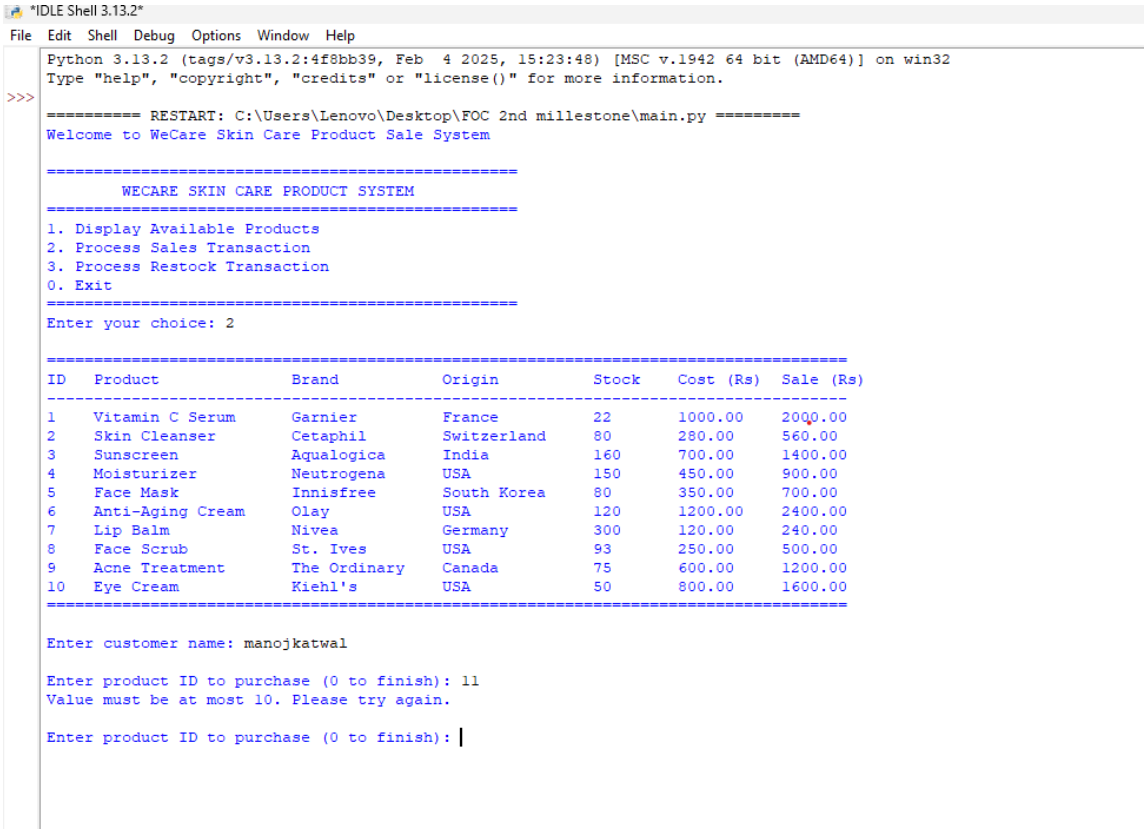


Figure 11:User entering product id that doesnt exist

9 Conclusion

The WeCare Skin product sale system project helps me to build my concept regarding python file handling, function, loops. Through the development of this system I learned how to deal with real word problem scenario, building logic to add “Buy 3 get 1 Free” policy, invoice generation and restocking the product. Overall project helped me to build my problem-solving skill and gain practical knowledge to build a business related software.

10 Appendix

1. Read module

```
"""
Read Module for WeCare Skin Care Product Sale System
This module handles all reading operations from files.
"""


def read_products():
    """
    Read product data from the products.txt file and return as a list of dictionaries.
    Each dictionary represents a product with keys: name, brand, quantity, cost_price, origin
    """
    products = []
    try:
        with open("products.txt", "r") as file:
            for line in file:
                line = line.strip()
                if line:  # Skip empty lines
                    parts = line.split(", ")
                    if len(parts) == 5:
                        product = {
                            "name": parts[0],
                            "brand": parts[1],
                            "quantity": int(parts[2]),
                            "cost_price": float(parts[3]),
                            "origin": parts[4]
                        }
                        products.append(product)
                    else:
                        print(f"Warning: Skipping invalid line format: {line}")
        return products
```

```
    except FileNotFoundError:
        print("Error: products.txt file not found!")
        return []
    except Exception as e:
        print(f"Error reading products file: {e}")
        return []

def display_products(products):
    if not products:
        print("No products available!")
        return

    print("\n" + "=" * 85)
    print(f"{'ID':<4} {'Product':<20} {'Brand':<15} {'Origin':<15} {'Stock':<8}
{'Cost (Rs)':<10} {'Sale (Rs)':<10}")
    print("-" * 85)

    index = 1
    for product in products:
        selling_price = product['cost_price'] * 2
        print(f"{index:<4}      {product['name']:<20}      {product['brand']:<15}
{product['origin']:<15} "
              f"{product['quantity']:<8}                {product['cost_price']:<10.2f}
{selling_price:<10.2f}")
        index += 1

    print("=" * 85)
```

2. Write module

```
"""
Write Module for WeCare Skin Care Product Sale System
This module handles all writing operations to files.
"""
from datetime import datetime

def write_products(products):
    """
    Write the updated product list back to the products.txt file.
    """
    try:
        with open("products.txt", "w") as file:
            for product in products:
                line = f"{product['name']}, {product['brand']}, {product['quantity']}, {product['cost_price']}, {product['origin']}\n"
                file.write(line)
        return True
    except Exception as e:
        print(f"Error writing to products file: {e}")
        return False


def generate_sales_invoice(customer_name, items, total_amount):
    """
    Generate a sales invoice as a text file.

    Args:
        customer_name: The name of the customer
        items: List of dictionaries with each item's details
        total_amount: The total sale amount
```

```
    """
    # Generate unique filename with timestamp
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"sale_{customer_name.replace(' ', '_')}_{timestamp}.txt"

    try:
        with open(filename, "w") as invoice:
            # Header
            invoice.write("=" * 60 + "\n")
            invoice.write("                WECARE SKIN CARE\n")
            invoice.write("                 SALES INVOICE\n")
            invoice.write("=" * 60 + "\n\n")

            # Invoice details
            invoice.write(f"Date:                {datetime.now().strftime('%d-%m-%Y
%H:%M:%S')}\n")
            invoice.write(f"Customer Name: {customer_name}\n")
            invoice.write(f"Invoice Number: INV-{timestamp}\n\n")

            # Table header
            invoice.write("-" * 60 + "\n")
            invoice.write(f"{'Product':<20}  {'Brand':<15}  {'Price':<10}  {'Qty':<5}
{'Free':<5} {'Amount':<10}\n")
            invoice.write("-" * 60 + "\n")

            # Items
            for item in items:
                invoice.write(f"{item['name']:<20}                {item['brand']:<15}
{item['selling_price']:<10.2f} "
                              f"{item['quantity']:<5}                {item['free']:<5}
{item['amount']:<10.2f}\n")
```

```
            # Footer
            invoice.write("-" * 60 + "\n")
            invoice.write(f"{'Total Amount:':<60} {total_amount:>10.2f}\n")
            invoice.write("=" * 60 + "\n\n")
            invoice.write("Thank you for shopping with WeCare!\n")
            invoice.write("Buy 3 Get 1 Free on all products!\n")

        print(f"Invoice generated successfully: {filename}")
        return filename
    except Exception as e:
        print(f"Error generating sales invoice: {e}")
        return None


def generate_restock_invoice(supplier_name, items, total_amount):
    """
    Generate a restock invoice as a text file.

    Args:
        supplier_name: The name of the supplier
        items: List of dictionaries with each item's details
        total_amount: The total restock amount
    """
    # Generate unique filename with timestamp
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"restock_{supplier_name.replace(' ', '_')}_{timestamp}.txt"

    try:
        with open(filename, "w") as invoice:
            # Header
```

```
            invoice.write("=" * 60 + "\n")
            invoice.write("                WECARE SKIN CARE\n")
            invoice.write("                 RESTOCK INVOICE\n")
            invoice.write("=" * 60 + "\n\n")

            # Invoice details
            invoice.write(f"Date:                {datetime.now().strftime('%d-%m-%Y
%H:%M:%S')}\n")
            invoice.write(f"Supplier Name: {supplier_name}\n")
            invoice.write(f"Purchase Order: PO-{timestamp}\n\n")

            # Table header
            invoice.write("-" * 60 + "\n")
            invoice.write(f"{'Product':<20}  {'Brand':<15}  {'Cost':<10}  {'Qty':<5}
{'Amount':<10}\n")
            invoice.write("-" * 60 + "\n")

            # Items
            for item in items:
                invoice.write(f"{item['name']:<20}                {item['brand']:<15}
{item['cost_price']:<10.2f} "
                              f"{item['quantity']:<5} {item['amount']:<10.2f}\n")

            # Footer
            invoice.write("-" * 60 + "\n")
            invoice.write(f"{'Total Amount:':<50} {total_amount:>10.2f}\n")
            invoice.write("=" * 60 + "\n\n")

        print(f"Restock invoice generated successfully: {filename}")
        return filename
    except Exception as e:
```

```
        print(f"Error generating restock invoice: {e}")
        return None
```

3. Operation module

```
"""
Operation Module for WeCare Skin Care Product Sale System
This module handles all business logic and operations.
"""
import read
import write


def get_valid_integer_input(prompt, min_value=0, max_value=None):
    """
    Get a valid integer input from the user.
    """
    while True:
        try:
            value = int(input(prompt))
            if value < min_value:
                print(f"Value must be at least {min_value}. Please try again.")
            elif max_value is not None and value > max_value:
                print(f"Value must be at most {max_value}. Please try again.")
            else:
                return value
        except ValueError:
            print("Please enter a valid number.")



def process_sales_transaction(products):
    """
    Process a sales transaction with the "buy 3 get 1 free" policy.
```

```
    """
    # Display products for sale
    read.display_products(products)

    # Get customer name
    while True:
        customer_name = input("\nEnter customer name: ").strip()
        if customer_name:
            break
        print("Customer name cannot be empty. Please try again.")

    sale_items = []
    total_amount = 0

    # Let customer select products
    while True:
        choice = get_valid_integer_input(f"\nEnter product ID to purchase (0 to finish): ", 0, len(products))

        if choice == 0:
            if not sale_items:
                print("No items selected. Transaction cancelled.")
                return False
            break

        product_idx = choice - 1
        product = products[product_idx]

        if product['quantity'] <= 0:
            print(f"Sorry, {product['name']} is out of stock.")
            continue
```

```
# Ask for quantity
quantity = get_valid_integer_input(
    f"Enter quantity for {product['name']} (max {product['quantity']}): ",
    1,
    product['quantity']
)

# Calculate selling price (200% markup)
selling_price = product['cost_price'] * 2

# Calculate free items based on the "buy 3 get 1 free" policy
free_items = quantity // 3
total_items = quantity + free_items

# Calculate amount for this item (excluding free items)
item_amount = quantity * selling_price

# Update total amount
total_amount += item_amount

# Add item to sale items
sale_items.append({
    'name': product['name'],
    'brand': product['brand'],
    'selling_price': selling_price,
    'quantity': quantity,
    'free': free_items,
    'amount': item_amount
})
```

```
            # Update product quantity
            products[product_idx]['quantity'] -= total_items

            print(f"Added {quantity} (+ {free_items} free) {product['name']} to cart.")

    # Generate invoice
    invoice_path = write.generate_sales_invoice(customer_name, sale_items, total_amount)

    if invoice_path:
        # Update product quantities in the file
        if write.write_products(products):
            print(f"\nTransaction completed successfully. Total amount: Rs. {total_amount:.2f}")
            print(f"Customer gets a total of {sum(item['free'] for item in sale_items)} free items!")
            return True

    print("Transaction failed.")
    return False


def process_restock_transaction(products):
    """
    Process a restock transaction.
    """
    # Display current products
    read.display_products(products)

    # Get supplier name
    while True:
```

```
            supplier_name = input("\nEnter supplier name: ").strip()
            if supplier_name:
                break
            print("Supplier name cannot be empty. Please try again.")

        restock_items = []
        total_amount = 0

        # Let admin select products to restock
        while True:
            choice = get_valid_integer_input(f"\nEnter product ID to restock (0 to finish): ", 0, len(products))

            if choice == 0:
                if not restock_items:
                    print("No items selected. Restock cancelled.")
                    return False
                break

            product_idx = choice - 1
            product = products[product_idx]

            # Ask for quantity
            quantity = get_valid_integer_input(f"Enter quantity to restock for {product['name']}: ", 1)

            # Ask if cost price has changed
            new_cost_price = product['cost_price']
            price_changed = input(f"Has the cost price changed? (Current: Rs. {new_cost_price:.2f}) (y/n): ").lower()
```

```
            if price_changed == 'y':
                while True:
                    try:
                        new_cost_price = float(input(f"Enter new cost price for {product['name']}: "))
                        if new_cost_price <= 0:
                            print("Cost price must be positive.")
                        else:
                            break
                    except ValueError:
                        print("Please enter a valid number.")

            # Calculate amount for this item
            item_amount = quantity * new_cost_price

            # Update total amount
            total_amount += item_amount

            # Add item to restock items
            restock_items.append({
                'name': product['name'],
                'brand': product['brand'],
                'cost_price': new_cost_price,
                'quantity': quantity,
                'amount': item_amount
            })

            # Update product quantity and cost price if changed
            products[product_idx]['quantity'] += quantity
            products[product_idx]['cost_price'] = new_cost_price
```

```
            print(f"Added {quantity} {product['name']} to restock list.")

        # Generate invoice
        invoice_path = write.generate_restock_invoice(supplier_name, restock_items, total_amount)

        if invoice_path:
            # Update product quantities and prices in the file
            if write.write_products(products):
                print(f"\nRestock completed successfully. Total amount: Rs. {total_amount:.2f}")
                return True

        print("Restock operation failed.")
        return False
```

4. Main module

```
Main Module for WeCare Skin Care Product Sale System
This is the entry point for the application.
"""
import read
import write
import operation



def display_menu():
    """
    Display the main menu options.
    """
    print("\n" + "=" * 50)
```

```
    print("       WECARE SKIN CARE PRODUCT SYSTEM")
    print("=" * 50)
    print("1. Display Available Products")
    print("2. Process Sales Transaction")
    print("3. Process Restock Transaction")
    print("0. Exit")
    print("=" * 50)


def main():
    """
    Main function to run the program.
    """
    print("Welcome to WeCare Skin Care Product Sale System")

    while True:
        # Read the latest product data
        products = read.read_products()

        if not products:
            print("No products available. Please check products.txt file.")
            if input("Would you like to try again? (y/n): ").lower() != 'y':
                break
            continue

        # Display menu and get user choice
        display_menu()

        try:
            choice = int(input("Enter your choice: "))
```

```
            if choice == 0:
                print("Thank you for using WeCare Skin Care Product System. Goodbye!")
                break
            elif choice == 1:
                read.display_products(products)
            elif choice == 2:
                operation.process_sales_transaction(products)
            elif choice == 3:
                operation.process_restock_transaction(products)
            else:
                print("Invalid choice. Please try again.")

        except ValueError:
            print("Please enter a valid number.")

        # Ask if user wants to perform another operation
        if input("\nWould you like to perform another operation? (y/n): ").lower() != 'y':
            print("Thank you for using WeCare Skin Care Product System. Goodbye!")
            break


if __name__ == "__main__":
    main()
```